

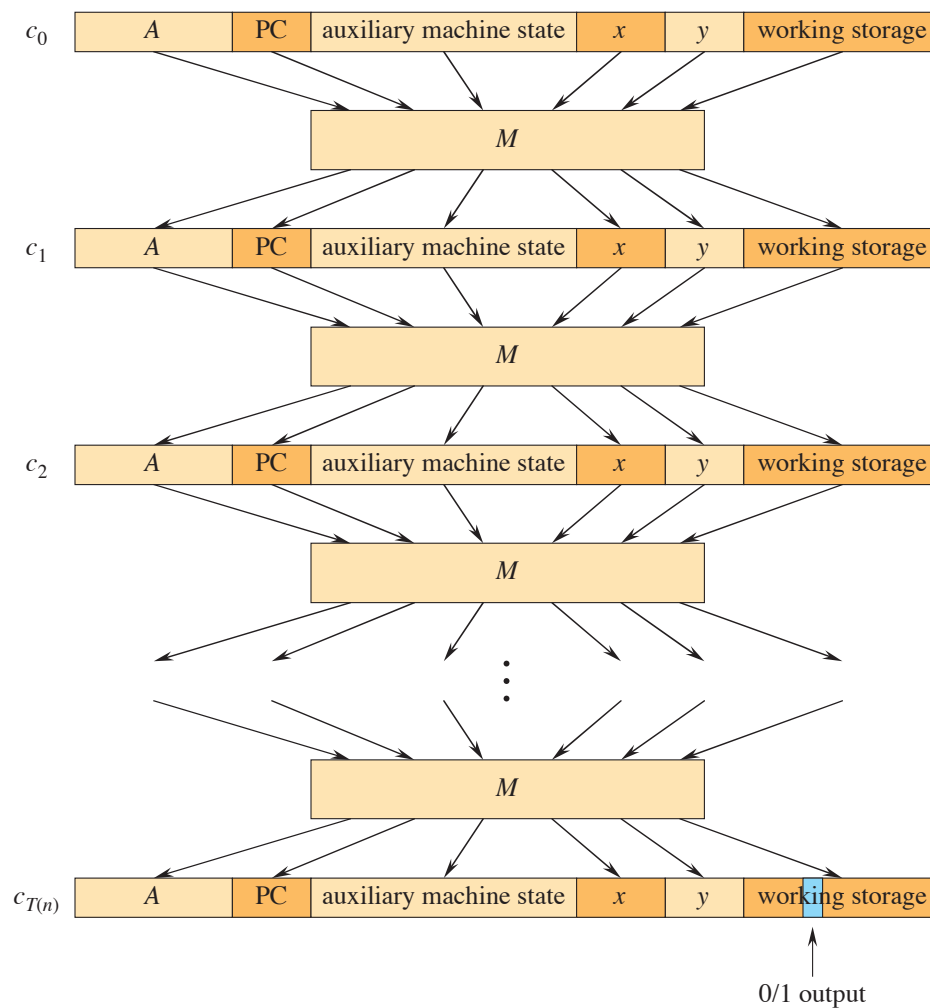


Figure 34.9 The sequence of configurations produced by an algorithm A running on an input x and certificate y . Each configuration represents the state of the computer for one step of the computation and, besides A , x , and y , includes the program counter (PC), auxiliary machine state, and working storage. Except for the certificate y , the initial configuration c_0 is constant. A boolean combinational circuit M maps each configuration to the next configuration. The output is a distinguished bit in the working storage.

certificate is $O(n^k)$. (The running time of A is actually a polynomial in the total input size, which includes both an input string and a certificate, but since the length of the certificate is polynomial in the length n of the input string, the running time is polynomial in n .)

The basic idea of the proof is to represent the computation of A as a sequence of configurations. As Figure 34.9 illustrates, consider each configuration as com-

prising a few parts: the program for A , the program counter and auxiliary machine state, the input x , the certificate y , and working storage. The combinational circuit M , which implements the computer hardware, maps each configuration c_i to the next configuration c_{i+1} , starting from the initial configuration c_0 . Algorithm A writes its output—0 or 1—to some designated location by the time it finishes executing. After A halts, the output value never changes. Thus, if the algorithm runs for at most $T(n)$ steps, the output appears as one of the bits in $c_{T(n)}$.

The reduction algorithm F constructs a single combinational circuit that computes all configurations produced by a given initial configuration. The idea is to paste together $T(n)$ copies of the circuit M . The output of the i th circuit, which produces configuration c_i , feeds directly into the input of the $(i + 1)$ st circuit. Thus, the configurations, rather than being stored in the computer's memory, simply reside as values on the wires connecting copies of M .

Recall what the polynomial-time reduction algorithm F must do. Given an input x , it must compute a circuit $C = f(x)$ that is satisfiable if and only if there exists a certificate y such that $A(x, y) = 1$. When F obtains an input x , it first computes $n = |x|$ and constructs a combinational circuit C' consisting of $T(n)$ copies of M . The input to C' is an initial configuration corresponding to a computation on $A(x, y)$, and the output is the configuration $c_{T(n)}$.

Algorithm F modifies circuit C' slightly to construct the circuit $C = f(x)$. First, it wires the inputs to C' corresponding to the program for A , the initial program counter, the input x , and the initial state of memory directly to these known values. Thus, the only remaining inputs to the circuit correspond to the certificate y . Second, it ignores all outputs from C' , except for the one bit of $c_{T(n)}$ corresponding to the output of A . This circuit C , so constructed, computes $C(y) = A(x, y)$ for any input y of length $O(n^k)$. The reduction algorithm F , when provided an input string x , computes such a circuit C and outputs it.

We need to prove two properties. First, we must show that F correctly computes a reduction function f . That is, we must show that C is satisfiable if and only if there exists a certificate y such that $A(x, y) = 1$. Second, we must show that F runs in polynomial time.

To show that F correctly computes a reduction function, suppose that there exists a certificate y of length $O(n^k)$ such that $A(x, y) = 1$. Then, upon applying the bits of y to the inputs of C , the output of C is $C(y) = A(x, y) = 1$. Thus, if a certificate exists, then C is satisfiable. For the other direction, suppose that C is satisfiable. Hence, there exists an input y to C such that $C(y) = 1$, from which we conclude that $A(x, y) = 1$. Thus, F correctly computes a reduction function.

To complete the proof sketch, we need to show that F runs in time polynomial in $n = |x|$. First, the number of bits required to represent a configuration is polynomial in n . Why? The program for A itself has constant size, independent of the length of its input x . The length of the input x is n , and the length of the certifi-

cate y is $O(n^k)$. Since the algorithm runs for at most $O(n^k)$ steps, the amount of working storage required by A is polynomial in n as well. (We implicitly assume that this memory is contiguous. Exercise 34.3-5 asks you to extend the argument to the situation in which the locations accessed by A are scattered across a much larger region of memory and the particular pattern of scattering can differ for each input x .)

The combinational circuit M implementing the computer hardware has size polynomial in the length of a configuration, which is $O(n^k)$, and hence, the size of M is polynomial in n . (Most of this circuitry implements the logic of the memory system.) The circuit C consists of $O(n^k)$ copies of M , and hence it has size polynomial in n . The reduction algorithm F can construct C from x in polynomial time, since each step of the construction takes polynomial time. ■

The language CIRCUIT-SAT is therefore at least as hard as any language in NP, and since it belongs to NP, it is NP-complete.

Theorem 34.7

The circuit-satisfiability problem is NP-complete.

Proof Immediate from Lemmas 34.5 and 34.6 and from the definition of NP-completeness. ■

Exercises

34.3-1

Verify that the circuit in Figure 34.8(b) is unsatisfiable.

34.3-2

Show that the $\leq_P$ relation is a transitive relation on languages. That is, show that if $L_1 \leq_P L_2$ and $L_2 \leq_P L_3$, then $L_1 \leq_P L_3$.

34.3-3

Prove that $L \leq_P \overline{L}$ if and only if $\overline{L} \leq_P L$.

34.3-4

Show that an alternative proof of Lemma 34.5 can use a satisfying assignment as a certificate. Which certificate makes for an easier proof?

34.3-5

The proof of Lemma 34.6 assumes that the working storage for algorithm A occupies a contiguous region of polynomial size. Where does the proof exploit this assumption? Argue that this assumption does not involve any loss of generality.

34.3-6

A language L is **complete** for a language class C with respect to polynomial-time reductions if $L \in C$ and $L' \leq_P L$ for all $L' \in C$. Show that $\emptyset$ and $\{0, 1\}^*$ are the only languages in P that are not complete for P with respect to polynomial-time reductions.

34.3-7

Show that, with respect to polynomial-time reductions (see Exercise 34.3-6), L is complete for NP if and only if $\overline{L}$ is complete for co-NP.

34.3-8

The reduction algorithm F in the proof of Lemma 34.6 constructs the circuit $C = f(x)$ based on knowledge of x , A , and k . Professor Sartre observes that the string x is input to F , but only the existence of A , k , and the constant factor implicit in the $O(n^k)$ running time is known to F (since the language L belongs to NP), not their actual values. Thus, the professor concludes that F cannot possibly construct the circuit C and that the language CIRCUIT-SAT is not necessarily NP-hard. Explain the flaw in the professor's reasoning.

34.4 NP-completeness proofs

The proof that the circuit-satisfiability problem is NP-complete showed directly that $L \leq_P \text{CIRCUIT-SAT}$ for every language $L \in \text{NP}$. This section shows how to prove that languages are NP-complete without directly reducing *every* language in NP to the given language. We'll explore examples of this methodology by proving that various formula-satisfiability problems are NP-complete. Section 34.5 provides many more examples.

The following lemma provides a foundation for showing that a given language is NP-complete.

Lemma 34.8

If L is a language such that $L' \leq_P L$ for some $L' \in \text{NPC}$, then L is NP-hard. If, in addition, we have $L \in \text{NP}$, then $L \in \text{NPC}$.

Proof Since L' is NP-complete, for all $L'' \in \text{NP}$, we have $L'' \leq_P L'$. By supposition, we have $L' \leq_P L$, and thus by transitivity (Exercise 34.3-2), we have $L'' \leq_P L$, which shows that L is NP-hard. If $L \in \text{NP}$, we also have $L \in \text{NPC}$. ■

In other words, by reducing a known NP-complete language L' to L , we implicitly reduce every language in NP to L . Thus, Lemma 34.8 provides a method for proving that a language L is NP-complete:

1. Prove $L \in \text{NP}$.
2. Prove that L is NP-hard:
 - a. Select a known NP-complete language L' .
 - b. Describe an algorithm that computes a function f mapping every instance $x \in \{0, 1\}^*$ of L' to an instance $f(x)$ of L .
 - c. Prove that the function f satisfies $x \in L'$ if and only if $f(x) \in L$ for all $x \in \{0, 1\}^*$.
 - d. Prove that the algorithm computing f runs in polynomial time.

This methodology of reducing from a single known NP-complete language is far simpler than the more complicated process of showing directly how to reduce from every language in NP. Proving CIRCUIT-SAT $\in$ NPC furnishes a starting point. Knowing that the circuit-satisfiability problem is NP-complete makes it much easier to prove that other problems are NP-complete. Moreover, as the catalog of known NP-complete problems grows, so will the choices for languages from which to reduce.

Formula satisfiability

To illustrate the reduction methodology, let's see an NP-completeness proof for the problem of determining whether a boolean *formula*, not a *circuit*, is satisfiable. This problem has the historical honor of being the first problem ever shown to be NP-complete.

We formulate the **(formula) satisfiability** problem in terms of the language SAT as follows. An instance of SAT is a boolean formula ϕ composed of

1. n boolean variables: $x_1, x_2, \dots, x_n$;
2. m boolean connectives: any boolean function with one or two inputs and one output, such as $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT), $\rightarrow$ (implication), $\leftrightarrow$ (if and only if); and
3. parentheses. (Without loss of generality, assume that there are no redundant parentheses, i.e., a formula contains at most one pair of parentheses per boolean connective.)

We can encode a boolean formula ϕ in a length that is polynomial in $n + m$. As in boolean combinational circuits, a **truth assignment** for a boolean formula ϕ

is a set of values for the variables of ϕ , and a *satisfying assignment* is a truth assignment that causes it to evaluate to 1. A formula with a satisfying assignment is a *satisfiable* formula. The satisfiability problem asks whether a given boolean formula is satisfiable, which we can express in formal-language terms as

$$\text{SAT} = \{ \langle \phi \rangle : \phi \text{ is a satisfiable boolean formula} \} .$$

As an example, the formula

$$\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$

has the satisfying assignment $\langle x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 1 \rangle$, since

$$\begin{aligned} \phi &= ((0 \rightarrow 0) \vee \neg((\neg 0 \leftrightarrow 1) \vee 1)) \wedge \neg 0 \\ &= (1 \vee \neg(1 \vee 1)) \wedge 1 \\ &= (1 \vee 0) \wedge 1 \\ &= 1 , \end{aligned} \tag{34.2}$$

and thus this formula ϕ belongs to SAT.

The naive algorithm to determine whether an arbitrary boolean formula is satisfiable does not run in polynomial time. A formula with n variables has 2^n possible assignments. If the length of $\langle \phi \rangle$ is polynomial in n , then checking every assignment requires $\Omega(2^n)$ time, which is superpolynomial in the length of $\langle \phi \rangle$. As the following theorem shows, a polynomial-time algorithm is unlikely to exist.

Theorem 34.9

Satisfiability of boolean formulas is NP-complete.

Proof We start by arguing that $\text{SAT} \in \text{NP}$. Then we prove that SAT is NP-hard by showing that $\text{CIRCUIT-SAT} \leq_P \text{SAT}$, which by Lemma 34.8 will prove the theorem.

To show that SAT belongs to NP, we show that a certificate consisting of a satisfying assignment for an input formula ϕ can be verified in polynomial time. The verifying algorithm simply replaces each variable in the formula with its corresponding value and then evaluates the expression, much as we did in equation (34.2) above. This task can be done in polynomial time. If the expression evaluates to 1, then the algorithm has verified that the formula is satisfiable. Thus, SAT belongs to NP.

To prove that SAT is NP-hard, we show that $\text{CIRCUIT-SAT} \leq_P \text{SAT}$. In other words, we need to show how to reduce any instance of circuit satisfiability to an instance of formula satisfiability in polynomial time. We can use induction to express any boolean combinational circuit as a boolean formula. We simply look at the gate that produces the circuit output and inductively express each of the

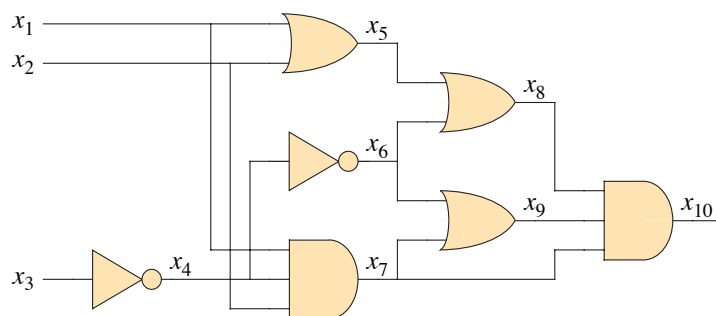


Figure 34.10 Reducing circuit satisfiability to formula satisfiability. The formula produced by the reduction algorithm has a variable for each wire in the circuit and a clause for each logic gate.

gate’s inputs as formulas. We then obtain the formula for the circuit by writing an expression that applies the gate’s function to its inputs’ formulas.

Unfortunately, this straightforward method does not amount to a polynomial-time reduction. As Exercise 34.4-1 asks you to show, shared subformulas—which arise from gates whose output wires have fan-out of 2 or more—can cause the size of the generated formula to grow exponentially. Thus, the reduction algorithm must be somewhat more clever.

Figure 34.10 illustrates how to overcome this problem, using as an example the circuit from Figure 34.8(a). For each wire x_i in the circuit C , the formula ϕ has a variable x_i . To express how each gate operates, construct a small formula involving the variables of its incident wires. The formula has the form of an “if and only if” ($\leftrightarrow$), with the variable for the gate’s output on the left and on the right a logical expression encapsulating the gate’s function on its inputs. For example, the operation of the output AND gate (the rightmost gate in the figure) is $x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9)$. We call each of these small formulas a *clause*.

The formula ϕ produced by the reduction algorithm is the AND of the circuit-output variable with the conjunction of clauses describing the operation of each gate. For the circuit in the figure, the formula is

$$\begin{aligned}
 \phi = & x_{10} \wedge (x_4 \leftrightarrow \neg x_3) \\
 & \wedge (x_5 \leftrightarrow (x_1 \vee x_2)) \\
 & \wedge (x_6 \leftrightarrow \neg x_4) \\
 & \wedge (x_7 \leftrightarrow (x_1 \wedge x_2 \wedge x_4)) \\
 & \wedge (x_8 \leftrightarrow (x_5 \vee x_6)) \\
 & \wedge (x_9 \leftrightarrow (x_6 \vee x_7)) \\
 & \wedge (x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9)) .
 \end{aligned}$$

Given a circuit C , it is straightforward to produce such a formula ϕ in polynomial time.

Why is the circuit C satisfiable exactly when the formula ϕ is satisfiable? If C has a satisfying assignment, then each wire of the circuit has a well-defined value, and the output of the circuit is 1. Therefore, when wire values are assigned to variables in ϕ , each clause of ϕ evaluates to 1, and thus the conjunction of all evaluates to 1. Conversely, if some assignment causes ϕ to evaluate to 1, the circuit C is satisfiable by an analogous argument. Thus, we have shown that $\text{CIRCUIT-SAT} \leq_p \text{SAT}$, which completes the proof. ■

3-CNF satisfiability

Reducing from formula satisfiability gives us an avenue to prove many problems NP-complete. The reduction algorithm must handle any input formula, though, and this requirement can lead to a huge number of cases to consider. Instead, it is usually simpler to reduce from a restricted language of boolean formulas. Of course, the restricted language must not be polynomial-time solvable. One convenient language is 3-CNF satisfiability, or 3-CNF-SAT.

In order to define 3-CNF satisfiability, we first need to define a few terms. A *literal* in a boolean formula is an occurrence of a variable (such as x_1) or its negation ($\neg x_1$). A *clause* is the OR of one or more literals, such as $x_1 \vee \neg x_2 \vee \neg x_3$. A boolean formula is in *conjunctive normal form*, or *CNF*, if it is expressed as an AND of clauses, and it's in *3-conjunctive normal form*, or *3-CNF*, if each clause contains exactly three distinct literals.

For example, the boolean formula

$$(x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_3 \vee x_2 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee \neg x_4)$$

is in 3-CNF. The first of its three clauses is $(x_1 \vee \neg x_1 \vee \neg x_2)$, which contains the three literals x_1 , $\neg x_1$, and $\neg x_2$.

The language 3-CNF-SAT consists of encodings of boolean formulas in 3-CNF that are satisfiable. The following theorem shows that a polynomial-time algorithm that can determine the satisfiability of boolean formulas is unlikely to exist, even when they are expressed in this simple normal form.

Theorem 34.10

Satisfiability of boolean formulas in 3-conjunctive normal form is NP-complete.

Proof The argument from the proof of Theorem 34.9 to show that $\text{SAT} \in \text{NP}$ applies equally well here to show that $3\text{-CNF-SAT} \in \text{NP}$. By Lemma 34.8, therefore, we need only show that $\text{SAT} \leq_p 3\text{-CNF-SAT}$.

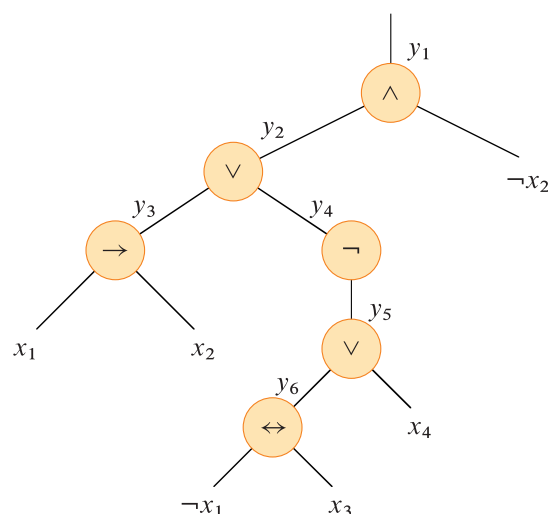


Figure 34.11 The tree corresponding to the formula $\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$.

We break the reduction algorithm into three basic steps. Each step progressively transforms the input formula ϕ closer to the desired 3-conjunctive normal form.

The first step is similar to the one used to prove $\text{CIRCUIT-SAT} \leq_P \text{SAT}$ in Theorem 34.9. First, construct a binary “parse” tree for the input formula ϕ , with literals as leaves and connectives as internal nodes. Figure 34.11 shows such a parse tree for the formula

$$\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2. \quad (34.3)$$

If the input formula contains a clause such as the OR of several literals, use associativity to parenthesize the expression fully so that every internal node in the resulting tree has just one or two children. The binary parse tree is like a circuit for computing the function.

Mimicking the reduction in the proof of Theorem 34.9, introduce a variable y_i for the output of each internal node. Then rewrite the original formula ϕ as the AND of the variable at the root of the parse tree and a conjunction of clauses describing the operation of each node. For the formula (34.3), the resulting expression is

$$\begin{aligned} \phi' = & y_1 \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_2)) \\ & \wedge (y_2 \leftrightarrow (y_3 \vee y_4)) \\ & \wedge (y_3 \leftrightarrow (x_1 \rightarrow x_2)) \\ & \wedge (y_4 \leftrightarrow \neg y_5) \\ & \wedge (y_5 \leftrightarrow (y_6 \vee x_4)) \\ & \wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3)). \end{aligned}$$

y_1	y_2	x_2	$(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	1

Figure 34.12 The truth table for the clause $(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$.

The formula ϕ' thus obtained is a conjunction of clauses ϕ'_i , each of which has at most three literals. These clauses are not yet ORs of three literals.

The second step of the reduction converts each clause ϕ'_i into conjunctive normal form. Construct a truth table for ϕ'_i by evaluating all possible assignments to its variables. Each row of the truth table consists of a possible assignment of the variables of the clause, together with the value of the clause under that assignment. Using the truth-table entries that evaluate to 0, build a formula in *disjunctive normal form* (or *DNF*)—an OR of ANDs—that is equivalent to $\neg\phi'_i$. Then negate this formula and convert it into a CNF formula ϕ''_i by using *DeMorgan's laws* for propositional logic,

$$\neg(a \wedge b) = \neg a \vee \neg b ,$$

$$\neg(a \vee b) = \neg a \wedge \neg b ,$$

to complement all literals, change ORs into ANDs, and change ANDs into ORs.

In our example, the clause $\phi'_1 = (y_1 \leftrightarrow (y_2 \wedge \neg x_2))$ converts into CNF as follows. The truth table for ϕ'_1 appears in Figure 34.12. The DNF formula equivalent to $\neg\phi'_1$ is

$$(y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2) .$$

Negating and applying DeMorgan's laws yields the CNF formula

$$\begin{aligned} \phi''_1 = & (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \\ & \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2) , \end{aligned}$$

which is equivalent to the original clause ϕ'_1 .

At this point, each clause ϕ'_i of the formula ϕ' has been converted into a CNF formula ϕ''_i , and thus ϕ' is equivalent to the CNF formula ϕ'' consisting of the conjunction of the ϕ''_i . Moreover, each clause of ϕ'' has at most three literals.